

TestSprite Backend Testing Product Specification

Project: Hello Staff - Hospitality Hiring SaaS

Purpose: Comprehensive backend QA for authentication, user roles, jobs, applications, dashboards, admin controls, data validation, and security behavior.

1. Test Environment

Field	Value
Base URL	http://localhost:3000
Path	/
Authentication Type	Basic - username and password
Username	wallykht1@gmail.com
Password	Wael@2026
Primary Objective	Run full backend regression tests against the local Hello Staff server.
Expected Output	A clear TestSprite report showing passed tests, failed tests, reproduction steps, request/response details, and severity.

Important security note: These credentials are only for the TestSprite test account. Rotate the password after testing if the account has any production-level access.

2. Product Context

Hello Staff is a SaaS hiring and staffing platform for the hospitality sector. It connects cafes and restaurants with candidates looking for jobs in roles such as barista, chef, waiter, cashier, host, cleaner, kitchen helper, and restaurant manager.

The backend should support multiple user roles, secure authentication, job posting, candidate profiles, applications, employer approval workflows, and admin oversight. The test goal is to verify backend correctness, data integrity, authorization, validation, and error handling.

3. User Roles to Validate

Role	Expected Permissions
Candidate / Worker	Can register/login, complete profile, upload or submit CV information, browse jobs, save jobs, apply to jobs, and track application status.
Employer Owner	Can manage company profile, create jobs, view applicants, approve/reject candidates, manage managers, and access employer dashboard data.
Employer Manager	Can assist with job and candidate workflows according to permissions assigned by the owner.
Platform Admin	Can review platform activity, manage users, monitor jobs, approve or restrict employers if implemented, and access platform-level dashboard data.

4. Backend Testing Scope

4.1 Authentication and Session Management

- Validate successful login with the provided Basic Auth test account.
- Validate failed login for wrong password, empty email, malformed email, and missing password.
- Validate protected endpoints reject unauthenticated requests.
- Validate logout/session invalidation behavior if the backend supports sessions or tokens.
- Validate rate limiting or lockout behavior if implemented.

4.2 User Registration and Profile APIs

- Create candidate, employer owner, employer manager, and admin-like users only where the system allows it.
- Validate required fields, email uniqueness, password requirements, phone format, role selection, and profile completion state.
- Validate profile update endpoints preserve role boundaries and reject unauthorized field changes.
- Validate CV/profile data can be created, updated, retrieved, and deleted according to permissions.

4.3 Job Posting APIs

- Validate employer users can create job posts with title, role type, location, employment type, salary/range if available, description, requirements, and status.
- Validate required-field errors and invalid enum values for job type, job status, employment type, and experience level.
- Validate candidates can list and view active jobs.
- Validate candidates cannot create, edit, or delete jobs.
- Validate employers cannot modify jobs belonging to another employer.

4.4 Application Workflow APIs

- Validate candidates can apply to an active job once only.
- Validate duplicate applications are rejected cleanly.
- Validate employers can view applicants for their own jobs.
- Validate approval, rejection, shortlist, and pending statuses if implemented.
- Validate candidates can view their own application status but not other users applications.

4.5 Dashboard and Analytics APIs

- Validate candidate dashboard returns saved jobs, applications, profile completion, and relevant notifications if available.
- Validate employer dashboard returns own jobs, applicants, open positions, pending approvals, and recent activity.
- Validate admin dashboard returns platform-level metrics only to admin users.
- Validate dashboard data is scoped by organization/user and does not leak cross-tenant data.

4.6 Admin and Moderation APIs

- Validate admin-only endpoints reject candidate and employer access.
- Validate admin can list users, jobs, employers, and platform activity if these APIs exist.
- Validate admin actions such as approving, suspending, deleting, or flagging entities if implemented.
- Validate audit trail or updated timestamps are written when admin actions occur.

4.7 Database and Supabase Integration

- Validate create/read/update/delete flows write to the expected tables.
- Validate foreign keys, unique constraints, not-null constraints, and status enums.
- Validate row-level security behavior: users only access records they own or are authorized to view.

- Validate timestamps, created_by, updated_by, organization_id, and role fields are consistent where implemented.

4.8 Error Handling and Security

- Validate backend returns proper HTTP status codes: 200/201, 400, 401, 403, 404, 409, and 500 when applicable.
- Validate errors are useful but do not expose secrets, stack traces, database keys, SQL errors, or private environment variables.
- Validate input sanitization against SQL injection-like strings, script tags, oversized payloads, and malformed JSON.
- Validate CORS behavior for expected origins if backend config exists.
- Validate file upload restrictions if CV or profile image upload exists: file type, size, and safe storage path.

5. Critical Test Scenarios

Scenario	Action	Expected Result
Auth Smoke Test	Use Basic Auth credentials and confirm backend accepts authenticated requests.	Pass with correct status code, correct response body, correct database state, and no unauthorized data exposure.
Candidate Happy Path	Create or use a candidate, complete profile, browse active jobs, save a job, apply once, and view application status.	Pass with correct status code, correct response body, correct database state, and no unauthorized data exposure.
Employer Happy Path	Create or use an employer owner, create job, update job, view applicants, approve/reject candidate, and verify dashboard reflects changes.	Pass with correct status code, correct response body, correct database state, and no unauthorized data exposure.
Permission Boundary Test	Attempt candidate actions as employer, employer actions as candidate, and admin actions as non-admin. All unauthorized actions must fail.	Pass with correct status code, correct response body, correct database state, and no unauthorized data exposure.
Cross-Tenant Data Leak Test	Create data under employer A and verify employer B cannot read or mutate it.	Pass with correct status code, correct response body, correct database state, and no unauthorized data exposure.
Validation Test	Send missing fields, invalid enums, invalid emails, long text, malformed JSON, duplicate records, and unsupported file types.	Pass with correct status code, correct response body, correct database state, and no unauthorized data exposure.
Regression Test	Run all available endpoints and confirm no 500 errors occur during normal business flows.	Pass with correct status code, correct response body, correct database state, and no unauthorized data exposure.

6. Acceptance Criteria

- All core backend endpoints are discovered and tested by TestSprite where reachable from http://localhost:3000/.
- Authentication works with the provided test credentials and fails safely for invalid credentials.
- Role-based authorization is enforced consistently across candidate, employer owner, employer manager, and platform admin flows.
- CRUD operations for users, profiles, jobs, and applications work without data corruption.

- No endpoint exposes secrets, stack traces, service role keys, private database errors, or other sensitive internals.
- All failed tests include reproducible request payloads, response bodies, status codes, and suggested severity.

7. TestSprite Instructions

1. Use the credentials in Section 1 for Basic Authentication.
2. Start from the local server root path and discover available backend routes automatically where possible.
3. Prioritize backend API behavior over frontend visual testing.
4. Generate a full regression report covering authentication, authorization, validation, business workflows, database consistency, and security issues.
5. For each failure, provide the failing endpoint, request method, payload, response status, response body, severity, and recommended fix.

8. Out of Scope for This Test Run

- Payment gateway testing unless payment APIs already exist.
- Production environment testing.
- Real user data testing.
- Frontend animation or visual polish testing.
- Third-party integrations unless configured locally.

9. Final Note for TestSprite

This document is intended to guide a full backend QA pass for the Hello Staff SaaS application. Treat the platform as a multi-role, multi-tenant hospitality hiring product. The highest priority is to catch backend logic failures, authorization leaks, broken business workflows, unsafe validation, and database inconsistency before launch.